

```

import os
import logging
from datetime import datetime
from telegram import Update, ReplyKeyboardMarkup, ReplyKeyboardRemove
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    filters, ContextTypes, ConversationHandler
)

logging.basicConfig(level=logging.INFO)
TOKEN = os.environ.get("BOT_TOKEN", "8917175933:AAGkwJnpCSvLMH3J1NA__QRWBeJPmeJ81

# States
(FLIGHT_NUM, DATE, CARRIER, ORIGIN, BLOCK_ON, STAND,
 AC_TYPE, PAX, BUS_COUNT, LATE_BUS, PREV_BUS, DELAY_MIN,
 SUBJECT, EXTRA, ACTIONS) = range(15)

ARABIC_MONTHS = [
    'يناير', 'فبراير', 'مارس', 'أبريل', 'مايو', 'يونيو',
    'يوليو', 'أغسطس', 'سبتمبر', 'أكتوبر', 'نوفمبر', 'ديسمبر'
]

def format_date(date_str):
    try:
        d = datetime.strptime(date_str, "%d/%m/%Y")
        return f"{d.day} {ARABIC_MONTHS[d.month]} {d.year}"
    except:
        return date_str

def build_report(d):
    summary = (
        f"قبل شركة {d['bus_count']} تمت متابعة الرحلة ميدانياً، وتم دعم الرحلة بـ"
        f"كانت جميع الباصات جاهزة ووصلت في الوقت المطلوب"
    )
    if d.get('late_bus') and d['late_bus'].lower() != 'لا':
        summary += (
            f"، باستثناء الباص رقم {d['late_bus']}"
            f"دقيقة {d['delay_min']} الذي تأخر لمدة"
            f"بعد مغادرة الباص رقم {d['prev_bus']}"
            f"مما أدى إلى انتظار بعض الركاب خارج الطائرة،"
        )
    else:
        summary += "."

```

```

if d.get('extra') and d['extra'].lower() != 'لا':
    summary += f"\n{d['extra']}"

if d.get('actions') and d['actions'].lower() != 'لا':
    actions = "\n".join(f"• {a.strip()}" for a in d['actions'].split('\n')) if
else:
    actions = "• تم توثيق الحالة ومتابعتها ميدانياً أثناء التشغيل."
    if d.get('late_bus') and d['late_bus'].lower() != 'لا':
        actions += f"\n• لباس رقم SAAS على شركة Inspection Report سوف يتم رفع

report = f"""
تقرير ميداني
{"-" * 35}
الرحلة      : {d['flight_num']}
التاريخ      : {format_date(d['date'])}
الناقل      : {d['carrier']}
القادمة من   : {d['origin']}
Block On     : {d['block_on']}
الموقف (Stand) : {d['stand']}
نوع الطائرة  : {d['ac_type']}
عدد الركاب    : {d['pax']} راكب
{"-" * 35}
الموضوع:
{d['subject']}

ملخص الحالة التشغيلية:
{summary}

الإجراء المتخذ:
{actions}
{"-" * 35}
"".strip()
return report

async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data.clear()
    await update.message.reply_text(
        "✈️ أهلاً! سأسألك عن بيانات الرحلة خطوة بخطوة\n\n*موّلد التقرير الميداني*
        parse_mode="Markdown"
    )
    return FLIGHT_NUM

async def get_flight_num(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['flight_num'] = update.message.text.strip()
    today = datetime.now().strftime("%d/%m/%Y")
    await update.message.reply_text(f"📅 *{today}*\n*التاريخ؟*")
    return DATE

```

```

async def get_date(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['date'] = update.message.text.strip()
    await update.message.reply_text("🏢 *الناقل*\nمثال: Saudi Airlines", parse_mode=ParseMode.MARKDOWN)
    return CARRIER

async def get_carrier(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['carrier'] = update.message.text.strip()
    await update.message.reply_text("✈️ *القادمة من*\nمثال: JED", parse_mode=ParseMode.MARKDOWN)
    return ORIGIN

async def get_origin(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['origin'] = update.message.text.strip()
    await update.message.reply_text("🕒 *Block On*\nمثال: 21:50", parse_mode=ParseMode.MARKDOWN)
    return BLOCK_ON

async def get_block_on(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['block_on'] = update.message.text.strip()
    await update.message.reply_text("🅑 *الموقف (Stand)*\nمثال: E25C", parse_mode=ParseMode.MARKDOWN)
    return STAND

async def get_stand(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['stand'] = update.message.text.strip()
    await update.message.reply_text("✈️ *نوع الطائرة*\nمثال: B777", parse_mode=ParseMode.MARKDOWN)
    return AC_TYPE

async def get_ac_type(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['ac_type'] = update.message.text.strip()
    await update.message.reply_text("👤 *عدد الركاب (PAX)*\nمثال: 353", parse_mode=ParseMode.MARKDOWN)
    return PAX

async def get_pax(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['pax'] = update.message.text.strip()
    await update.message.reply_text("🚌 *عدد الباصات المخصصة*\nمثال: 7", parse_mode=ParseMode.MARKDOWN)
    return BUS_COUNT

async def get_bus_count(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['bus_count'] = update.message.text.strip()
    await update.message.reply_text("🚌 *رقم الباص المتأخر*\nمثال: 7", parse_mode=ParseMode.MARKDOWN)
    return LATE_BUS

async def get_late_bus(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['late_bus'] = update.message.text.strip()
    if ctx.user_data['late_bus'].lower() == 'لا':
        ctx.user_data['prev_bus'] = ''
        ctx.user_data['delay_min'] = ''
        await update.message.reply_text("✍️ *نقل الركاب الأخير*\nالموضوع:", parse_mode=ParseMode.MARKDOWN)

```

```

        return SUBJECT
    await update.message.reply_text("🚌 *مثال: SR146*", parse_mode=ParseMode.MARKDOWN)
    return PREV_BUS

async def get_prev_bus(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['prev_bus'] = update.message.text.strip()
    await update.message.reply_text("⌚ *مثال: مدة التأخير بالدقائق؟*", parse_mode=ParseMode.MARKDOWN)
    return DELAY_MIN

async def get_delay_min(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['delay_min'] = update.message.text.strip()
    await update.message.reply_text("✍️ *ال: تأخر باص نقل الركاب الأخير*\nالموضوع؟*", parse_mode=ParseMode.MARKDOWN)
    return SUBJECT

async def get_subject(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['subject'] = update.message.text.strip()
    await update.message.reply_text("📄 *إذا ما في إضافات*\nتفاصيل إضافية للملخص؟*", parse_mode=ParseMode.MARKDOWN)
    return EXTRA

async def get_extra(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['extra'] = update.message.text.strip()
    await update.message.reply_text("⚡ *للإجراءات الافتراضية*\nالإجراءات المتخذة؟*", parse_mode=ParseMode.MARKDOWN)
    return ACTIONS

async def get_actions(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    ctx.user_data['actions'] = update.message.text.strip()
    report = build_report(ctx.user_data)
    await update.message.reply_text(
        f"✅ *تم إنشاء التقرير*\n{report}",
        parse_mode="Markdown"
    )
    await update.message.reply_text("🔄 /start لتقرير جديد أرسل")
    return ConversationHandler.END

async def cancel(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("❌ /start من جديد تم الإلغاء. أرسل")
    return ConversationHandler.END

def main():
    app = Application.builder().token(TOKEN).build()
    conv = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            FLIGHT_NUM: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_flight_num)],
            DATE: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_date)],
            CARRIER: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_carrier)],
            ORIGIN: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_origin)]
        }
    )

```

```

        BLOCK_ON: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_bloc
        STAND: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_stan
        AC_TYPE: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_ac_t
        PAX: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_pax)
        BUS_COUNT: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_bus_
        LATE_BUS: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_late
        PREV_BUS: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_prev
        DELAY_MIN: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_dela
        SUBJECT: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_subj
        EXTRA: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_extr
        ACTIONS: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_acti
    },
    fallbacks=[CommandHandler("cancel", cancel)],
)
app.add_handler(conv)
print("✅ البوت شغال!")
app.run_polling()

if __name__ == "__main__":
    main()

```